src/viz.py · fk, to_px MMA3001 DeepONet project

fk (forward kinematics)

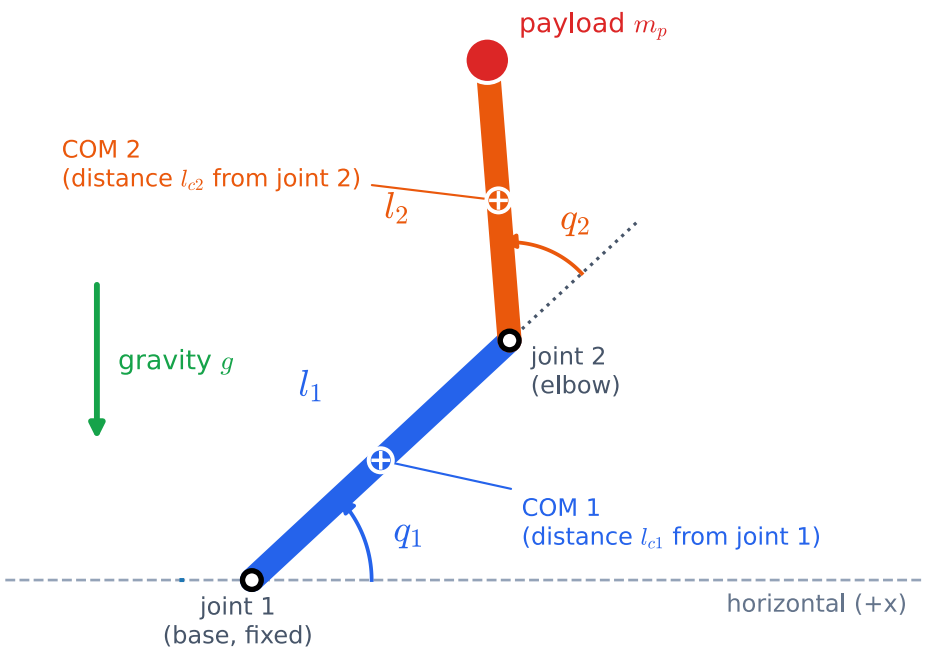
From joint angles to positions of the elbow and tip, plus the metre-to-pixel conversion used to draw the arm

In this document

1. The question
2. Toolbox: polar to Cartesian
3. Elbow and tip
4. Workspace and two postures
5. Metres to pixels
6. Worked example
7. Code-maths dictionary

The set-up: a two-link robot arm in a vertical plane

Everything in this project describes one physical object: a planar robot arm with two rigid links, two motorised joints, bolted to a wall at the origin. It moves in a *vertical* plane, so gravity matters. At the tip of link 2 we may clip on an extra weight (the **payload**). Each joint has a little friction that slows it down in proportion to how fast it spins.



The arm. Blue = link 1, orange = link 2. The angle q_1 is measured from the horizontal; the angle q_2 is measured from the *straight-on extension* of link 1 (dotted), not from the horizontal. The dots with a white cross are the links' centres of mass (COM).

Conventions (fixed everywhere in the project): the state is $x = [q_1, q_2, \dot{q}_1, \dot{q}_2]$; gravity points in the $-y$ direction; potential energy is zero at the height of joint 1; positive angles and torques are anticlockwise.

Symbol	Meaning	Value in the code	Unit
$l_1,\ l_2$	link lengths (joint to joint)	0.5, 0.4	m

$m_1,\ m_2$	link masses (without payload)	2.0, 1.5	kg
$l_{c1},\ l_{c2}$	distance from a link's joint to its own centre of mass (COM)	0.25, 0.2 (middle of each link)	m
$I_1,\ I_2$	moment of inertia of a link about its own COM	$ml^2/12$ (uniform rod)	kg m^2
g	gravitational acceleration	9.81	m/s^2
m_p	point-mass payload at the tip of link 2 (varies per sample)	0 to 1 (dataset), up to 1.5 out-of-range	kg
b	viscous joint friction coefficient (varies per sample)	0.05 to 0.5	N m s/rad
$q_1,\ q_2$	joint angles: q_1 from the horizontal, q_2 relative to link 1	state	rad
$\dot{q}_1,\ \dot{q}_2$	joint angular velocities (a dot means d/dt)	state	rad/s
$\tau_1,\ \tau_2$	motor torques applied at joints 1 and 2 (the inputs)	$A\sin(2\pi ft+\phi)$	N m

1 The question this function answers

The solver works with joint *angles*. But a picture of the arm needs *positions* on the screen: where is the elbow, where is the tip? Going from angles to positions is called **forward kinematics** ("kinematics" = geometry of motion, ignoring forces). The visualiser (`src/viz.py`) uses it to draw the arm; the same geometry (with centre-of-mass distances l_c in place of the link lengths l) is the one used for heights in the energy.

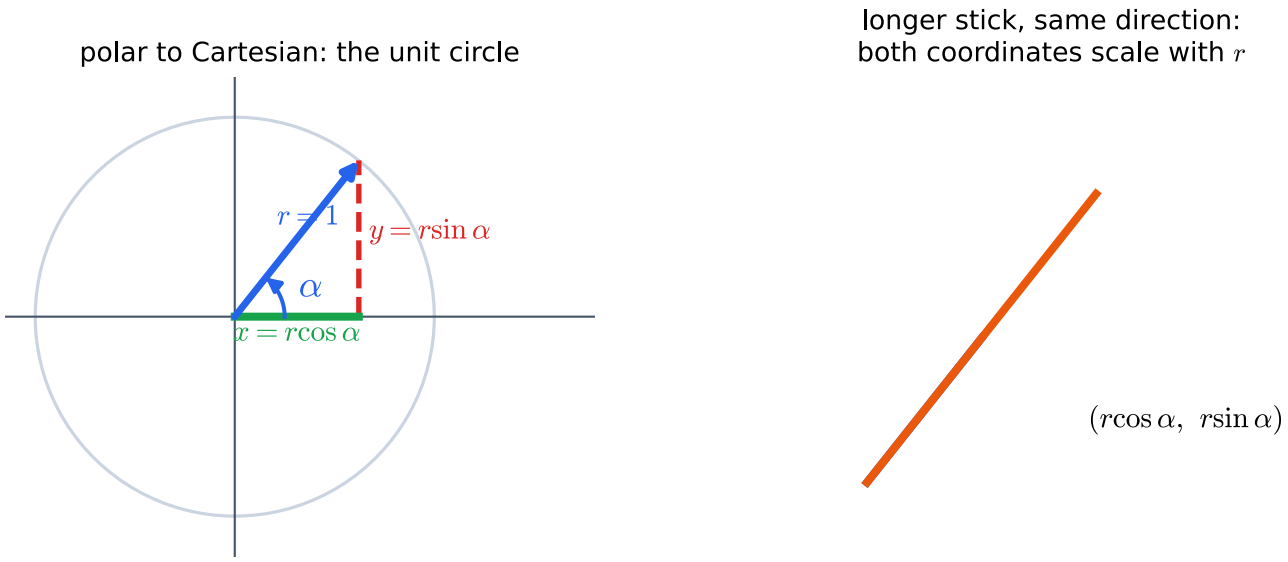
```
def fk(q, p: ArmParams):
    """Joint positions (elbow, tip) in metres, y up."""
    e = np.array([p.l1 * np.cos(q[0]), p.l1 * np.sin(q[0])])
    tip = e + np.array([p.l2 * np.cos(q[0] + q[1]), p.l2 * np.sin(q[0] + q[1])])
    return e, tip
```

2 Toolbox: polar to Cartesian

You can describe a point in the plane either by (x, y) or by "how far, and in which direction": a distance r and an angle α measured anticlockwise from the $+x$ axis. The conversion comes from the definition of sine and cosine on the unit circle (a circle of radius 1):

$$x = r\cos\alpha, \qquad y = r\sin\alpha.$$

For $r = 1$ the point on the circle is $(\cos\alpha, \sin\alpha)$; for a longer stick of the same direction, both coordinates simply scale with r. Equivalently, in a right triangle $\cos\alpha = \text{adjacent}/\text{hypotenuse}$ and $\sin\alpha = \text{opposite}/\text{hypotenuse}$. (Angles in radians: π rad $= 180^\circ$.)



Polar to Cartesian. The green horizontal leg is $r\cos\alpha$ and the red vertical leg is $r\sin\alpha$.

3 Building the position of the elbow and the tip

Step 1: the elbow

Link 1 is a stick of length l_1 fixed at the origin, pointing in direction q_1 (that is how q_1 is defined). Its far end, the elbow, is therefore

$$\mathbf{e} = \big(l_1\cos q_1,\ l_1\sin q_1\big).$$

Step 2: the direction of link 2 — why the angles add

q_2 is measured *relative* to link 1. If link 1 were along the x axis ($q_1 = 0$), link 2 would point at angle q_2. Now rotate the entire arm anticlockwise by q_1 (turn the base joint): *every* direction increases by q_1, so link 2 points at the absolute angle

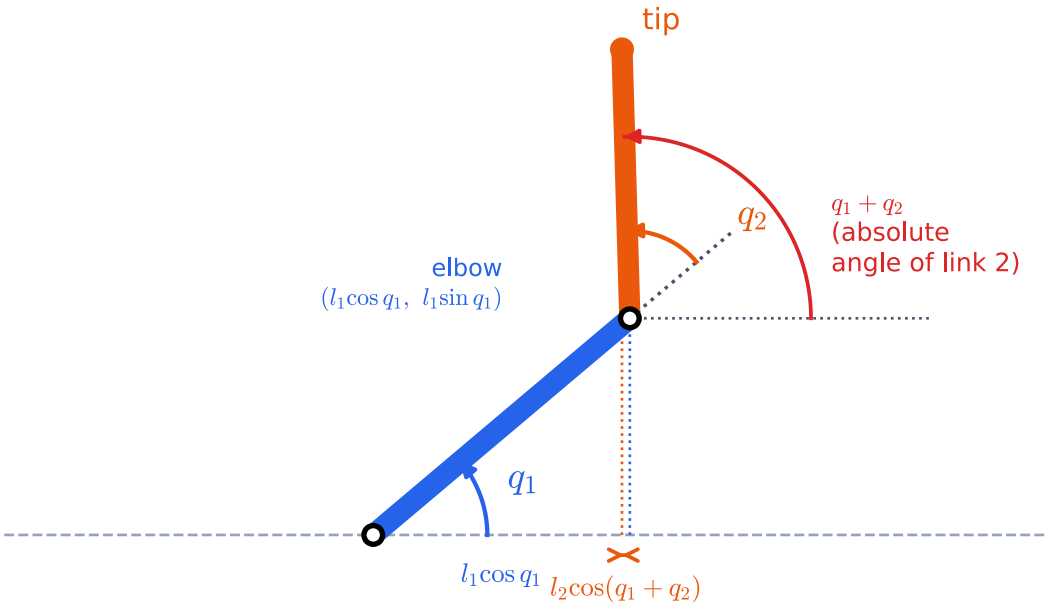
$$\theta_2 = q_1 + q_2.$$

Step 3: the tip

Start at the elbow and walk a distance l_2 in direction θ_2. Positions add like vectors (tip = elbow + link-2 vector):

$$\mathbf{tip} = \mathbf{e} + l_2\big(\cos(q_1+q_2),\ \sin(q_1+q_2)\big) = \Big(l_1\cos q_1 + l_2\cos(q_1+q_2),\ \ l_1\sin q_1 + l_2\sin(q_1+q_2)\Big).$$

These are exactly the two lines of the function: `e = ...` and `tip = e + ...`.



Constructing the tip. The elbow is a stick of length l_1 at angle q_1. The tip is the elbow plus a stick of length l_2 at the *absolute* angle $q_1 + q_2$ (red arc, measured from the horizontal). Dotted lines project onto the x axis, giving the $\cos$ terms.

Compact version with rotation matrices (optional)

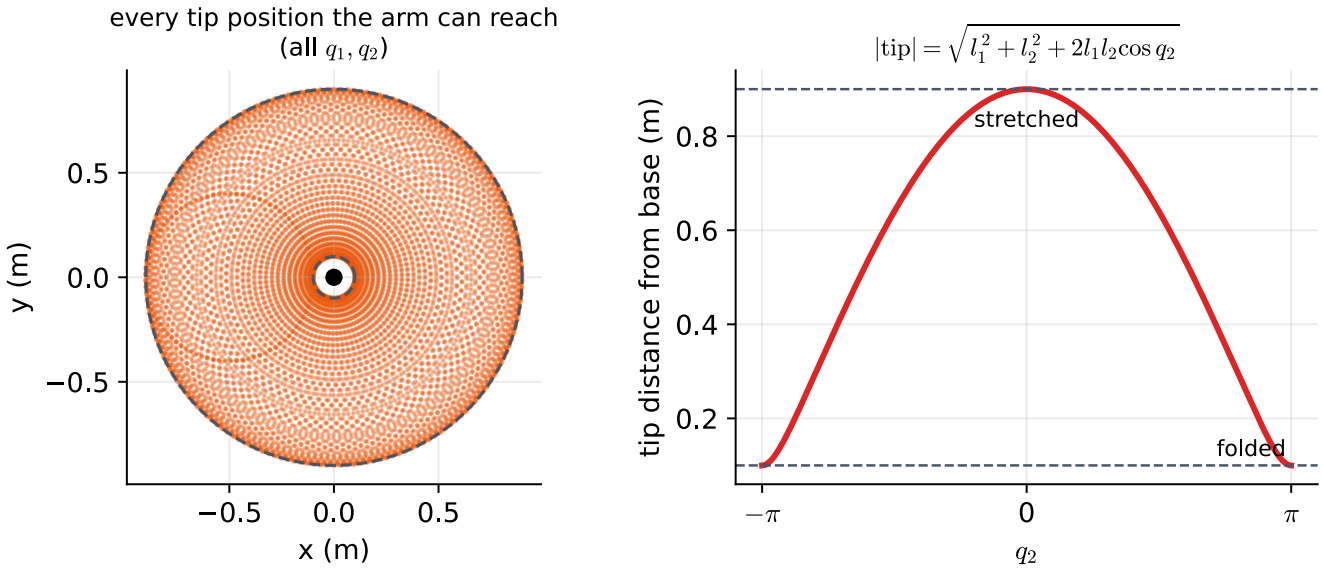
Turning a vector anticlockwise by an angle θ is multiplication by $R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$. The same formulas read $\mathbf{tip} = R(q_1)\binom{l_1}{0} + R(q_1+q_2)\binom{l_2}{0}$, because $R(\theta)\binom{l}{0} = l\binom{\cos\theta}{\sin\theta}$. The "angles add" fact is the identity $R(q_1)R(q_2) = R(q_1+q_2)$ (this is the sine/cosine addition formulas in disguise).

4 What the arm can reach: the workspace

The squared distance of the tip from the base follows from squaring the two coordinates and using $\cos^2 + \sin^2 = 1$ plus the cosine subtraction formula (same trick as in *mass_matrix*):

$$|\mathbf{tip}|^2 = l_1^2 + l_2^2 + 2\,l_1 l_2 \cos q_2.$$

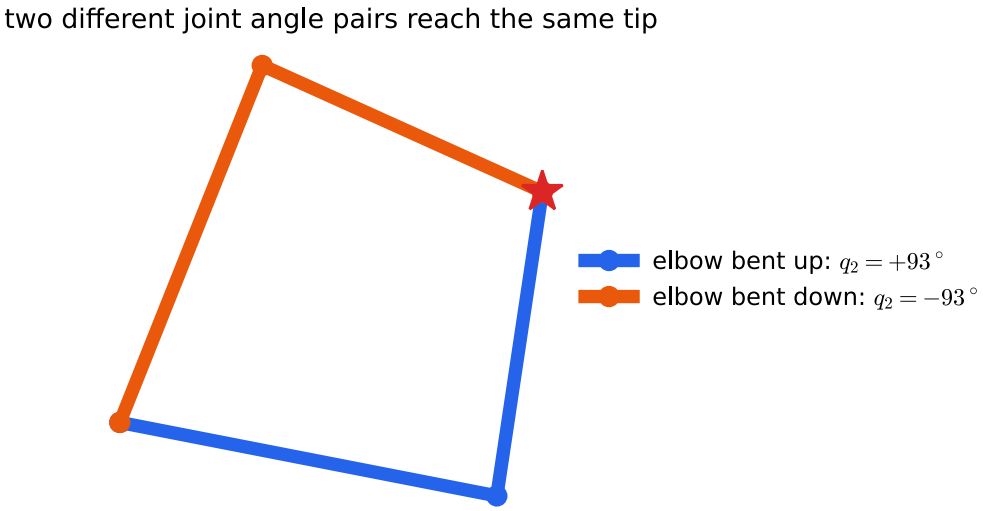
It depends only on the elbow angle: stretched ($q_2 = 0$) gives the maximum $l_1 + l_2 = 0.9$ m; folded back ($q_2 = \pm\pi$) gives the minimum $|l_1 - l_2| = 0.1$ m. So the reachable set is a ring (annulus) around the base. That is also the law-of-cosines picture for the triangle (base, elbow, tip).



The workspace. Left: tip positions for a grid of all joint angles fill the ring between radius 0.1 m and 0.9 m. Right: tip distance from the base as a function of q_2 alone.

Bonus: going backwards (inverse kinematics) — two answers

Given a target tip T, solve $|T|^2 = l_1^2 + l_2^2 + 2l_1 l_2 \cos q_2$ for q_2: $\cos q_2 = \dfrac{|T|^2 - l_1^2 - l_2^2}{2l_1 l_2}$. Since cos is even, **both $+q_2$ and $-q_2$ work**: the elbow can bend either way (two "postures"). Then q_1 follows from the direction of T minus a correction. The code never needs this (the simulation starts from given angles), but it explains why the forward map angles→position is not one-to-one.



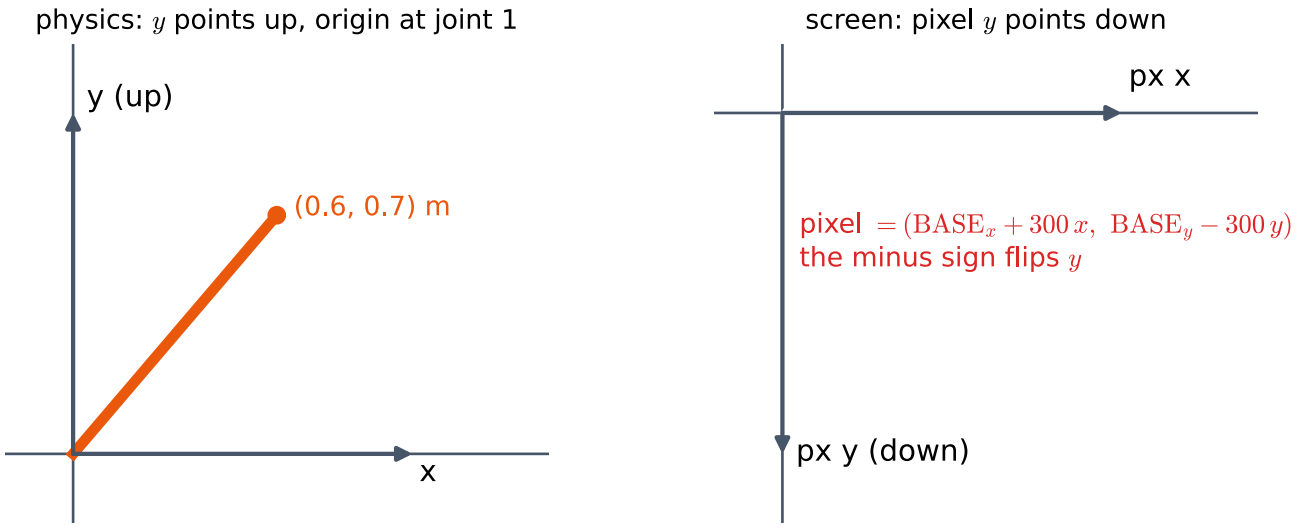
Two arm postures, one tip. A star marks the target; blue and orange are the two joint-angle solutions.

5 From metres to pixels (`to_px`)

The physics uses metres with y pointing up. A computer screen counts in pixels from the top-left corner with y pointing *down*. `to_px` converts, with the base joint drawn at pixel `BASE = (280, 330)` and a scale of `PX_PER_M = 300` pixels per metre:

$$\text{px} = \text{BASE}_x + 300\,x, \qquad \text{py} = \text{BASE}_y - 300\,y.$$

The minus sign flips the vertical direction. (A 0.5 m link is thus 150 pixels long.) `draw_arm` then just draws two line segments base→elbow→tip with these pixel positions; if there is a payload it adds a circle at the tip whose radius grows with m_p ($6 + 14\,m_p$ pixels).



Two coordinate systems. The same physical point has a positive y in the physics frame but a *smaller* pixel row (higher up the screen) in the screen frame.

One more tiny formula lives in the visualiser: `draw_torque` draws an arc whose sweep angle is $\mathrm{sweep} = \mathrm{clip}(\tau / A_{\max}, -1, 1) \cdot \pi$ with $A_{\max} = 15$ N m, i.e. a purely cosmetic linear scaling (full torque = half a circle). It is not physics.

6 Worked example with real numbers

Default arm, $q = (0.5,\ 0.8)$ rad. $\cos q_1 = 0.8776$, $\sin q_1 = 0.4794$; $q_1 + q_2 = 1.3$ rad: $\cos 1.3 = 0.2675$, $\sin 1.3 = 0.9636$.

point	calculation	position (m)
elbow	$(0.5 \cdot 0.8776,\ 0.5 \cdot 0.4794)$	(0.4388, 0.2397)
tip	elbow $+ (0.4 \cdot 0.2675,\ 0.4 \cdot 0.9636)$	(0.5458, 0.6251)
distance of tip	$\sqrt{0.5^2 + 0.4^2 + 2 \cdot 0.5 \cdot 0.4 \cos 0.8}$	0.8299 (direct: $\sqrt{x^2 + y^2} = 0.8299$) ✓
pixels	elbow: $(280 + 300x,\ 330 - 300y)$; tip likewise	elbow (411.6, 258.1), tip (443.7, 142.5)

7 Code ↔ maths dictionary

code	maths
`p.l1 * np.cos(q[0])`, `p.l1 * np.sin(q[0])`	elbow $= (l_1 \cos q_1,\ l_1 \sin q_1)$
`q[0] + q[1]`	absolute angle of link 2: $\theta_2 = q_1 + q_2$
`e + np.array([...])`	tip = elbow + link-2 vector
`BASE[1] - pt[1]*PX_PER_M`	flip y for the screen

CHECK YOURSELF

1. Arm horizontal and straight, $q = (0, 0)$: where is the tip? *Answer:* $(l_1 + l_2,\ 0) = (0.9,\ 0)$.
2. Arm pointing straight up with the elbow bent a right angle to the right ($q = (\pi/2,\ -\pi/2)$): tip? *Answer:* elbow $(0, 0.5)$; link 2 at absolute angle 0, so tip $= (0.4,\ 0.5)$.
3. Why does the elbow position not depend on q_2? *Answer:* the elbow is the end of link 1; q_2 only rotates link 2 about the elbow.